МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8

“Компьютерные науки и прикладная математика”

Кафедра 806

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

по учебной дисциплине «Операционные системы и архитектура компьютера» на тему «Управление процессами и обмен данными через каналы»

Вариант 12

Выполнил:

Юрин Матвей Юрьевич

М8О-206БВ-25

Проверил:

Ефимов Д.А.

Москва 2026

СОДЕРЖАНИЕ

1 Цель работы и задание....3

2 Сборка и запуск....3

3 Как работает программа....3

4 Основные системные вызовы....4

5 Строки произвольной длины....4

6 Проверка программы....5

7 Получение трассы....6

8 Исходный код....8

9 Вывод....13

10 Использование нейросетей....13

1 Цель работы и задание

Цель работы - научиться создавать дочерние процессы и передавать данные между ними через каналы. Для варианта 12 я сделал три отдельные программы: parent, child1 и child2.

По заданию родитель создаёт двух детей и принимает строки произвольной длины. Child1 переводит строку в верхний регистр. Child2 убирает все задвоенные пробелы. Готовая строка возвращается родителю, и он выводит её на экран. Обмен данными должен идти через pipe, а системные ошибки нужно обрабатывать.

2 Сборка и запуск

Я собирал все три программы отдельно. Эти команды выполняются из папки program:

```
gcc -Wall -Wextra -Wpedantic -std=c11 -D_POSIX_C_SOURCE=200809L child1.c -o child1
gcc -Wall -Wextra -Wpedantic -std=c11 -D_POSIX_C_SOURCE=200809L child2.c -o child2
gcc -Wall -Wextra -Wpedantic -std=c11 -D_POSIX_C_SOURCE=200809L parent.c -o parent
./parent
```

Запускать parent нужно из той же папки, где находятся child1 и child2. В коде используются пути ./child1 и ./child2. Сначала я проверил сборку на macOS, потом повторил её в Ubuntu 26.04.1 LTS в UTM. В Ubuntu использовался GCC 15.2.0. Предупреждений при сборке не было.

После ввода строк программу можно завершить через Ctrl+D на пустой строке. Это означает конец ввода, то есть EOF.

3 Как работает программа

Программа работает по такой схеме:

```
parent -> pipe1 -> child1 -> pipe2 -> child2 -> pipe3 -> parent
```

Сначала родитель создаёт три канала через pipe и двух детей через fork. В первом ребёнке запускается child1, во втором - child2. Для запуска используется execl: он заменяет программу внутри уже созданного процесса.

В каждом канале конец [0] нужен для чтения, а [1] - для записи. Родитель пишет исходную строку в pipe1[1], а child1 читает её из pipe1[0]. Затем child1 пишет строку в pipe2[1], а child2 читает её из pipe2[0]. После удаления лишних пробелов child2 пишет результат в pipe3[1], а родитель читает его из pipe3[0].

Чтобы дети могли пользоваться обычными stdin и stdout, я перенаправил их через dup2. В child1 stdin связан с pipe1[0], stdout - с pipe2[1]. В child2 stdin связан с pipe2[0], stdout - с pipe3[1]. Номера стандартных потоков: stdin - 0, stdout - 1, stderr - 2.

Лишние концы каналов закрываются через close. Это нужно для правильного EOF: чтение закончится только после закрытия всех копий конца записи. Если забыть закрыть лишний конец, процесс может продолжить ждать данные. В конце родитель закрывает pipe1[1] и ждёт обоих детей через waitpid.

4 Основные системные вызовы

Вызов	Что он делает в этой работе
pipe	Создаёт канал и возвращает два файловых дескриптора.
fork	Создаёт дочерний процесс.
dup2	Перенаправляет stdin и stdout ребёнка на нужные каналы.
execl	Запускает child1 или child2 внутри дочернего процесса.
write	Передаёт строку в канал.
close	Закрывает лишние и больше не нужные дескрипторы.
waitpid	Ждёт завершения детей и получает их статусы.

Файловый дескриптор - это номер открытого файла или канала в процессе. Он не содержит саму строку: через этот номер программа говорит системе, откуда читать и куда писать.

Результаты вызовов проверяются. При ошибке программа пишет сообщение в stderr и возвращает ненулевой код. Если write или waitpid прерван сигналом с ошибкой EINTR, вызов повторяется. SIGPIPE игнорируется, чтобы при обрыве канала можно было получить ошибку записи и вывести сообщение.

Дети отправляют результат через fwrite и fflush. Родитель тоже вызывает fflush после вывода каждой строки. Без этого данные могут остаться в буфере, и следующий процесс будет их ждать.

5 Строки произвольной длины

Раньше фиксированный массив мог ограничить строку 1024 символами. Для произвольной длины я использовал getline. В начале указатель равен NULL, а вместимость буфера равна 0:

```
char *line = NULL;
size_t capacity = 0;
ssize_t length = getline(&line, &capacity, stdin);
```

getline читает целую строку и сама выделяет или увеличивает память. &line нужен, чтобы функция могла записать новый адрес в указатель. &capacity нужен, чтобы она могла сообщить размер выделенного буфера.

capacity - это вместимость буфера в байтах, а length - длина реально прочитанной строки. Например, строка может занимать 6 байтов, а памяти под неё выделено 120. getline учитывает перенос строки, если он есть, но не включает завершающий нулевой байт в возвращаемую длину. После работы память освобождается через free.

В начале файлов указан _POSIX_C_SOURCE 200809L. Эта настройка позволяет заголовочным файлам объявить функции POSIX, в том числе getline и fdopen, при сборке в режиме C11.

Один write может записать только часть строки, поэтому родитель продолжает запись циклом, пока не отправит все байты. Результат из pipe3 он читает через getline, связав канал с потоком через fdopen. Если последняя строка не заканчивается переносом,

родитель закрывает pipe1[1] до ожидания ответа. Тогда дети получают EOF и могут закончить чтение этой строки.

Child1 вызывает toupper для каждого байта строки. Проверено преобразование латинских букв ASCII. Перевод регистра русских букв в UTF-8 эта реализация не выполняет. Child2 сокращает только обычные пробелы до одного; табуляции и переносы строк сохраняются.

6 Проверка программы

Я проверил программу на macOS и в Ubuntu. В каждом тесте весь результат сравнивался с ожидаемой строкой. На один запуск было отведено 15 секунд. Все проверки прошли, ошибок и зависаний не было.

Проверка	**Вход, байт**	**Выход, байт**	**Итог**
Обычная строка	22	16	Пройден
Без двойных пробелов	16	16	Пройден
Много пробелов подряд	88	6	Пройден
Три строки подряд	38	34	Пройден
Пустая строка	1	1	Пройден
Только пробелы	6	2	Пройден
5000 символов и перенос	5001	4287	Пройден
200001 символ и перенос	200002	200002	Пройден
Строка без переноса	13	9	Пройден
Сразу EOF	0	0	Пройден
Табуляции сохраняются	8	7	Пройден

В таблице длина включает перенос строки, если он есть. Кроме этих 11 проверок, я отдельно запустил child1 и child2. Оба теста тоже прошли.

Пример обычной строки:

```
Ввод:    hello     my   friend
Вывод:   HELLO MY FRIEND
```

Для длинного теста я взял первые 5000 байтов повторяющейся строки "ab cd " и добавил перенос. На выходе получилось 4287 байтов. Весь результат совпал с ожидаемым, поэтому уменьшение длины связано только с удалением лишних пробелов. В другом тесте я повторил "ab " 66667 раз и добавил перенос. Получилось 200002 байта и на входе, и на выходе. Строка не обрезалась.

Последняя строка "last   line" без переноса превратилась в "LAST LINE", тоже без переноса. Пустая строка осталась пустой, а пять пробелов сократились до одного.

Я также проверил запуск без файла child1, без файла child2 и ошибку вывода в /dev/full. Программа сообщила об ошибке и завершилась без зависания. Результат обычной строки появился ещё до отправки EOF. После EOF дочерние процессы завершились.

Через Valgrind в Ubuntu я проверил память всех трёх программ на обычной тестовой строке. Для каждого процесса получено 0 ошибок и 0 байтов в 0 блоках при выходе. Журнал проверки сохранён вместе с результатами тестов.

7 Получение трассы

Для просмотра системных вызовов я запустил программу через strace в Ubuntu:

```
strace -f -o trace.txt ./parent < trace-input.txt > trace-output.txt
```

В файле ввода была строка "hello my friend" с переносом. После её чтения наступил EOF. На выходе получилось "HELLO MY FRIEND". Флаг -f нужен, чтобы следить за родителем и обоими детьми. Полная трасса сохранена в results/trace.txt. Ниже приведены настоящие строки из этого запуска.

В трассе родитель имеет PID 3692, первый ребёнок - 3693, второй - 3694. Каналы получили дескрипторы [3,4], [5,6] и [7,8].

```
3692  pipe2([3, 4], 0)                  = 0
3692  pipe2([5, 6], 0)                  = 0
3692  pipe2([7, 8], 0)                  = 0
3692  clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0xe61612ecc030) = 3693
3693  dup3(3, 0, 0 <unfinished ...>
3692  clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD
<unfinished ...>
3693  <... dup3 resumed>)               = 0
3692  <... clone resumed>, child_tidptr=0xe61612ecc030) = 3694
3693  dup3(6, 1, 0 <unfinished ...>
3693  <... dup3 resumed>)               = 1
3693  close(3 <unfinished ...>
3692  close(3 <unfinished ...>
3693  <... close resumed>)              = 0
3692  <... close resumed>)              = 0
3694  dup3(5, 0, 0 <unfinished ...>
3694  <... dup3 resumed>)               = 0
3694  dup3(8, 1, 0 <unfinished ...>
3694  <... dup3 resumed>)               = 1
3694  close(3 <unfinished ...>
3694  <... close resumed>)              = 0
3693  execve("./child1", ["child1"], 0xffffd7ca3ce8 /* 10 vars */ <unfinished ...>
3693  <... execve resumed>)             = 0
3692  read(0 <unfinished ...>
3692  <... read resumed>, "hello     my   friend\n", 4096) = 22
3694  execve("./child2", ["child2"], 0xffffd7ca3ce8 /* 10 vars */ <unfinished ...>
3692  write(4, "hello     my   friend\n", 22) = 22
3694  <... execve resumed>)             = 0
3692  read(7 <unfinished ...>
3693  read(0 <unfinished ...>
3693  <... read resumed>, "hello     my   friend\n", 4096) = 22
3694  read(0 <unfinished ...>
3693  write(1, "HELLO     MY   FRIEND\n", 22 <unfinished ...>
3694  <... read resumed>, "HELLO     MY   FRIEND\n", 4096) = 22
3693  <... write resumed>)              = 22
3693  read(0 <unfinished ...>
3694  write(1, "HELLO MY FRIEND\n", 16) = 16
3692  <... read resumed>, "HELLO MY FRIEND\n", 4096) = 16
3694  read(0 <unfinished ...>
3692  write(1, "HELLO MY FRIEND\n", 16) = 16
```

```
3692  read(0, "", 4096)                 = 0
3692  close(4)                          = 0
3693  <... read resumed>, "", 4096)     = 0
3692  close(7 <unfinished ...>
3692  <... close resumed>)              = 0
3692  wait4(3693 <unfinished ...>
3694  <... read resumed>, "", 4096)     = 0
3693  +++ exited with 0 +++
3692  <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 3693
3692  wait4(3694 <unfinished ...>
3694  +++ exited with 0 +++
3692  <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 3694
3692  +++ exited with 0 +++
```

В коде используются функции библиотеки Си, а strace показывает системные вызовы ядра. Поэтому здесь pipe виден как pipe2, fork - как clone, dup2 - как dup3 с флагом 0, execl - как execve, waitpid - как wait4. Это нормально. Строки unfinished и resumed показывают начало и продолжение вызова, между которыми выполнялся другой процесс. Родитель и оба ребёнка завершились с кодом 0.

8 Исходный код

Ниже приведены текущие исходные файлы. Вся логика родителя находится в main. Программы собираются отдельно.

parent.c

```
#define _POSIX_C_SOURCE 200809L

#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    int pipe1[2] = {-1, -1};
    int pipe2[2] = {-1, -1};
    int pipe3[2] = {-1, -1};
    struct sigaction action = {0};
    action.sa_handler = SIG_IGN;
    if (sigemptyset(&action.sa_mask) == -1 || sigaction(SIGPIPE, &action, NULL) == -1)
{
        perror("sigaction");
        return 1;
    }

    if (pipe(pipe1) == -1 || pipe(pipe2) == -1 || pipe(pipe3) == -1) {
        perror("pipe");
        for (int i = 0; i < 2; i++) {
            if (pipe1[i] != -1) {
                close(pipe1[i]);
            }
            if (pipe2[i] != -1) {
                close(pipe2[i]);
            }
            if (pipe3[i] != -1) {
                close(pipe3[i]);
            }
        }
        return 1;
    }

    pid_t child1_pid = fork();
    if (child1_pid == -1) {
        perror("fork child1");
        for (int i = 0; i < 2; i++) {
            close(pipe1[i]);
            close(pipe2[i]);
            close(pipe3[i]);
        }
        return 1;
    }
    if (child1_pid == 0) {
        if (dup2(pipe1[0], STDIN_FILENO) == -1 || dup2(pipe2[1], STDOUT_FILENO) == -1)
{
            perror("dup2 child1");
            _exit(1);
        }

        int close_error = 0;
```

```
        for (int i = 0; i < 2; i++) {
            if (close(pipe1[i]) == -1) {
                close_error = 1;
            }
            if (close(pipe2[i]) == -1) {
                close_error = 1;
            }
            if (close(pipe3[i]) == -1) {
                close_error = 1;
            }
        }
        if (close_error) {
            perror("close");
            _exit(1);
        }

        execl("./child1", "child1", NULL);
        perror("execl child1");
        _exit(1);
    }

    pid_t child2_pid = fork();
    if (child2_pid == -1) {
        perror("fork child2");
        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
        close(pipe3[0]);
        close(pipe3[1]);
        waitpid(child1_pid, NULL, 0);
        return 1;
    }
    if (child2_pid == 0) {
        if (dup2(pipe2[0], STDIN_FILENO) == -1 || dup2(pipe3[1], STDOUT_FILENO) == -1)
{
            perror("dup2 child2");
            _exit(1);
        }

        int close_error = 0;
        for (int i = 0; i < 2; i++) {
            if (close(pipe1[i]) == -1) {
                close_error = 1;
            }
            if (close(pipe2[i]) == -1) {
                close_error = 1;
            }
            if (close(pipe3[i]) == -1) {
                close_error = 1;
            }
        }
        if (close_error) {
            perror("close");
            _exit(1);
        }

        execl("./child2", "child2", NULL);
        perror("execl child2");
        _exit(1);
    }
```

```
    int error = 0;
    if (close(pipe1[0]) == -1) {
        error = 1;
    }
    if (close(pipe2[0]) == -1) {
        error = 1;
    }
    if (close(pipe2[1]) == -1) {
        error = 1;
    }
    if (close(pipe3[1]) == -1) {
        error = 1;
    }
    if (error) {
        perror("close");
    }

    FILE *pipe3_input = fdopen(pipe3[0], "r");
    if (pipe3_input == NULL) {
        perror("fdopen");
        close(pipe1[1]);
        close(pipe3[0]);
        waitpid(child1_pid, NULL, 0);
        waitpid(child2_pid, NULL, 0);
        return 1;
    }

    char *line = NULL;
    char *result = NULL;
    size_t line_capacity = 0;
    size_t result_capacity = 0;
    ssize_t line_length;
    int pipe1_open = 1;

    while (!error && (line_length = getline(&line, &line_capacity, stdin)) != -1) {
        size_t written = 0;

        while (written < (size_t)line_length) {
            ssize_t count = write(pipe1[1], line + written, (size_t)line_length -
written);
            if (count == -1 && errno == EINTR) {
                continue;
            }
            if (count <= 0) {
                perror("write");
                error = 1;
                break;
            }
            written += (size_t)count;
        }
        if (error) {
            break;
        }

        if (line[line_length - 1] != '\n') {
            if (close(pipe1[1]) == -1) {
                perror("close pipe1");
                error = 1;
            }
            pipe1_open = 0;
        }
```

```
        ssize_t result_length = getline(&result, &result_capacity, pipe3_input);
        if (result_length == -1) {
            fprintf(stderr, "Cannot read result from child2\n");
            error = 1;
            break;
        }
        if (fwrite(result, 1, (size_t)result_length, stdout) != (size_t)result_length
|| fflush(stdout) == EOF) {
            perror("stdout");
            error = 1;
            break;
        }
    }

    if (!error && !feof(stdin)) {
        perror("stdin");
        error = 1;
    }

    free(line);
    free(result);
    if (pipe1_open && close(pipe1[1]) == -1) {
        perror("close pipe1");
        error = 1;
    }
    if (fclose(pipe3_input) == EOF) {
        perror("fclose pipe3");
        error = 1;
    }

    pid_t children[2] = {child1_pid, child2_pid};
    for (int i = 0; i < 2; i++) {
        int status;
        pid_t waited;
        do {
            waited = waitpid(children[i], &status, 0);
        } while (waited == -1 && errno == EINTR);

        if (waited == -1) {
            perror("waitpid");
            error = 1;
        } else if (!WIFEXITED(status) || WEXITSTATUS(status) != 0) {
            fprintf(stderr, "child%d failed\n", i + 1);
            error = 1;
        }
    }

    return error;
}
```

child1.c

```
#define _POSIX_C_SOURCE 200809L

#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    char *line = NULL;
    size_t capacity = 0;
    ssize_t length;
```

```
    int result = 0;

    while ((length = getline(&line, &capacity, stdin)) != -1) {
        for (ssize_t i = 0; i < length; i++) {
            line[i] = (char)toupper((unsigned char)line[i]);
        }

        if (fwrite(line, 1, (size_t)length, stdout) != (size_t)length ||
fflush(stdout) == EOF) {
            perror("stdout");
            result = 1;
            break;
        }
    }

    if (!result && !feof(stdin)) {
        perror("stdin");
        result = 1;
    }

    free(line);
    return result;
}
```

child2.c

```
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    char *line = NULL;
    size_t capacity = 0;
    ssize_t length;
    int result = 0;

    while ((length = getline(&line, &capacity, stdin)) != -1) {
        size_t output_length = 0;

        for (ssize_t i = 0; i < length; i++) {
            if (line[i] == ' ' && output_length > 0 && line[output_length - 1] == ' ')
{
                continue;
            }

            line[output_length++] = line[i];
        }

        if (fwrite(line, 1, output_length, stdout) != output_length || fflush(stdout)
== EOF) {
            perror("stdout");
            result = 1;
            break;
        }
    }

    if (!result && !feof(stdin)) {
        perror("stdin");
        result = 1;
    }
```

```
    free(line);
    return result;
}
```

9 Вывод

В этой работе я сделал три программы, которые обрабатывают строки и передают их через каналы. Я разобрал, как fork создаёт детей, exec запускает в них отдельные программы, а dup2 связывает стандартные потоки с pipe. Программа правильно обработала обычные и длинные строки и завершилась по EOF. Проверки на macOS и Ubuntu прошли. Через strace я увидел передачу данных и завершение всех трёх процессов.

10 Использование нейросетей

При выполнении лабораторной работы я использовал нейросети для помощи в разборе каналов и getline, проверке программы и оформлении отчёта.